Step 1 — A Single LCEL Chain That Answers One Question, No Tools — Hints

Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — The three pieces of a chain
- Hint 2 — The plan, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The three pieces of a chain

This step proves one thing: LangChain can turn a question into an answer. No tools, no loop.

An LCEL chain is three pieces joined with `|` (Doc05):

- a **prompt** — `ChatPromptTemplate.from_template("... {question}")`;
- a **model** — `ChatOpenAI(model="gpt-4o-mini", ...)`;
- a **parser** — `StrOutputParser()`, which turns the model's reply into plain text.

`chain.invoke({"question": "..."})` runs all three in order. The dictionary key must match the `{question}` placeholder exactly.

Split the pieces into the files they'll live in for the rest of the project:

- `models/llm.py` — `get_model(config) -> ChatOpenAI`, with `temperature=0` and `api_key=config.openai_api_key`.
- `prompts/direct.txt` — the prompt text, with a `{question}` placeholder.
- `utils/prompts.py` — Project 1's `load_prompt(name)`, with the folder changed to `src/prompts`.
- `chains/direct_answer.py` — `build_direct_chain(model)`: load the prompt, build the template, return `prompt | model | parser`.

Passing the model *into* `build_direct_chain()` (instead of building it inside) means every chain and agent later shares one model from `get_model()`.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
main():
    config = load_config()
    prompt = ChatPromptTemplate.from_template("Answer: {question}")
    model = ChatOpenAI(model="gpt-4o-mini", api_key=...)
    chain = prompt | model | StrOutputParser()
    print(chain.invoke({"question": "What is the capital of France?"}))

run it from the repo root:  python main.py
```

```
main():
    try: config = load_config()
    except MissingConfigError: print a "Setup problem" line, exit(1)
    logger = get_logger(__name__)
    model = get_model(config)
    chain = build_direct_chain(model)
    loop:
        question = input("You: ").strip()
        "/quit" -> stop,  "" -> skip
        try: answer = chain.invoke({"question": question})
        except openai.AuthenticationError: log, print, exit(1)
        print("Answer (from memory):", answer)
```

Try "What's the weather in Lahore right now?". The answer is a guess — that's expected, and it's why Step 2 exists.

Full solution: Show me the solution